

TITLE 제목

common.js 내 이벤트/유틸리티. ↔ manifest 간 관계 파악

Continued from page:

< 함수 네이밍 관련 >

1) 공식 이벤트 (리버스 호출됨). → onHomepage

2) 비동기 유틸 → buildHomeCard

- Manifest 내 runFunctions2 정의하는 함수.
- Simple Trigger / Installable Trigger
- Webapp 이벤트

src
↳ app-script/

common.js ← 공통 이벤트
gmail.js ← Gmail "
drive.js ← drive "
calendar.js ← calendar "
query.js ← 서버 통신
util/ ← 유틸리티

"manifest + 드래기 (특수함수)."
↳ 이 runFunctions2를 불러옴...

< 공식 이벤트 >

- onHomepage
- onGmail Message
- onGmail/compose
- onDriveItemsSelected

notion 단편화.
and 정리

< to do >

함수 네이밍 통일 &
구조 이해하며
전체 다시 짜보기
↑ 가능할까?

• parquet/prompt
• mail2json.js
• app.py. 등 서버 구현
↳ routing.

< Gmail Add-on event >

Gmail Add-on
mail
...
1) 아이콘 클릭.
↳ onHomepage() → buildHomeCard()

2) 메일 클릭.

↳ onGmail Message(e)
↳ buildGmail/MessageCard()

KNOX

Ftext) ← Header.
1) onHomepage.
2) onGmail Message(e)
3) onSmartSearch()

webapp URL
↳ onSyncNewOnly()
↳ onSyncAll()

onApplyLabelToMessage()
1) onHomepage.
2) onGmail Message(e)
3) onSmartSearch()
4) onUploadSingleMessage()

< 현재 gmail.js 코드 >

[onSyncNewOnly()
onSyncAll()

return runSync(false);
∴ 현재는 '전체갱신'만 가능.

↓
구조보안 필요!
(03/04)

RECORDED BY

DATE 기록일

WITNESSED BY

DATE 점검일

2026.0

TITLE 제목

Parquet (파케이).

Continued from page :

(Hadoop 생태계에서 주로 사용됨.)

· 정의: 컬럼 기반 데이터 저장 방식.

· 특징:

① 높은 압축률

A1	B1	C1
A2	B2	C2
A3	B3	C3

< column 기반 format >

→ [A1 | B1 | C1 | A2 | B2 | C2 | A3 | B3 | C3]

(e.g. MySQL)

왜? 같은 행에는 (종종) 유사한 데이터가 나열되기 때문 ⇒ 컬럼 단위의 압축.

② 효율적인 데이터 추가.

⇒ 디스크 I/O

(∵ 새로운 기록은 끝에 추가 → 고품 쓰기 가능)

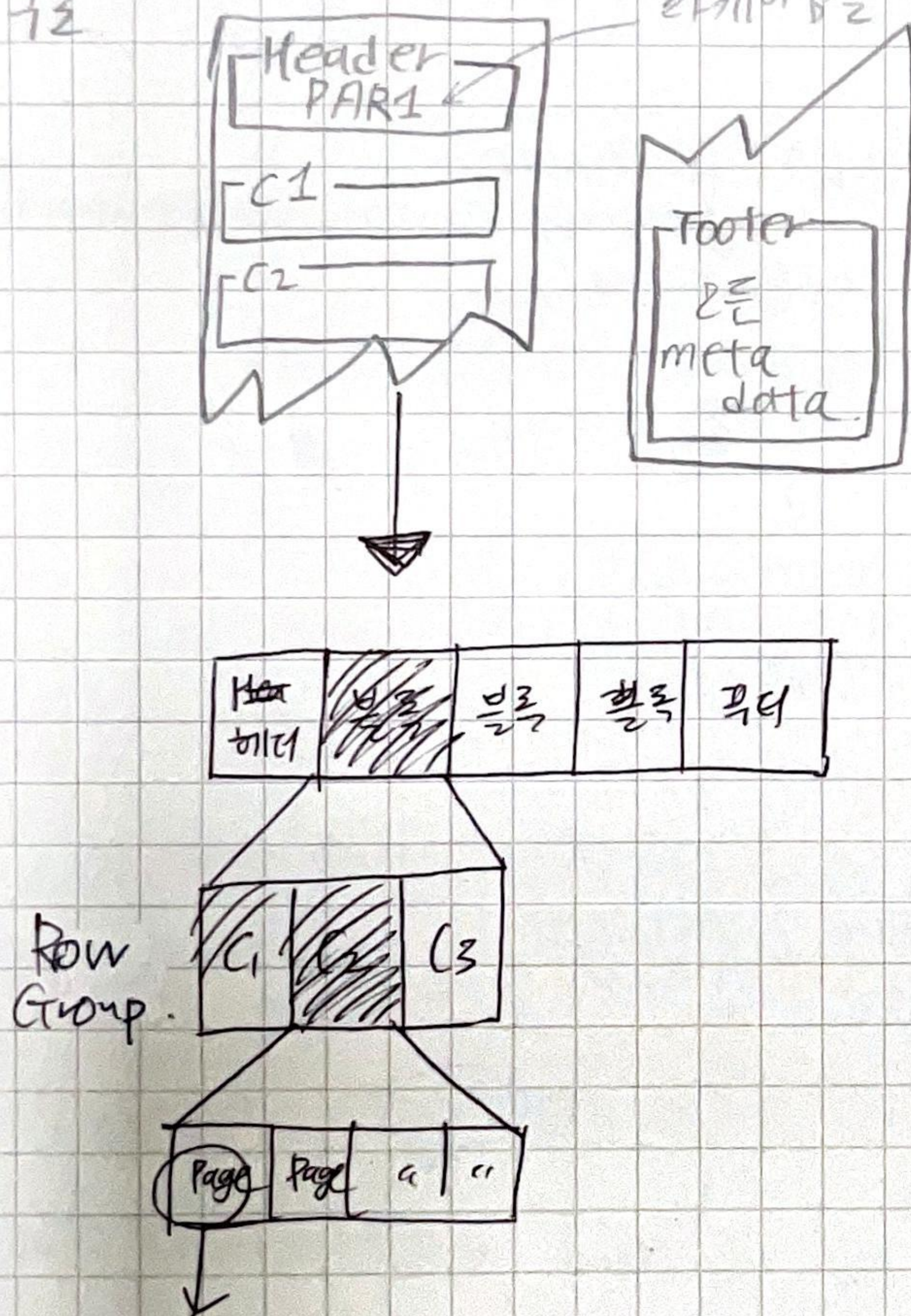
데이터를 미리 컬럼 단위로 정리해 둔다.

컬럼과 데이터들은 동일 타입 →

magic

③ 컬럼 데이터 리딩이
직접한 링크를 사용 가능

· 내부 구조



데이터 최소 단위.

∴ Page는 동일 컬럼 (동일 data type)
데이터만 존재.

∴ 인코딩/압축 시, Page 단위로 수행.

parquet/

(Raw Data)

- ~~input/~~ ← 원본 데이터 레이어
 mail-latest.txt 저장도

- cache/ ← 재계산 방지 중간데이터들

- community-reporting/
- entity-extraction/
- extract-graph
- summarize-description/
- text-embedding/

- logs/ ← 실행 로그

- output/ ← 최종 산출물 저장도

- prompts. ← LLM 동작 정보.

- settings.yaml ← 전역 설정 파일

Gmail Add-on

mail-latest.txt

↓ read
mail2json.py↓ write
graphml-data.json

GraphRAG

pipe line

↓
parquet/ 저장.

Continued to page :

RECORDED BY 기록자

최지영.

DATE 기록일

2026.03.04(수)

SIGNATURE 서명

최지영.

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

TITLE 제목

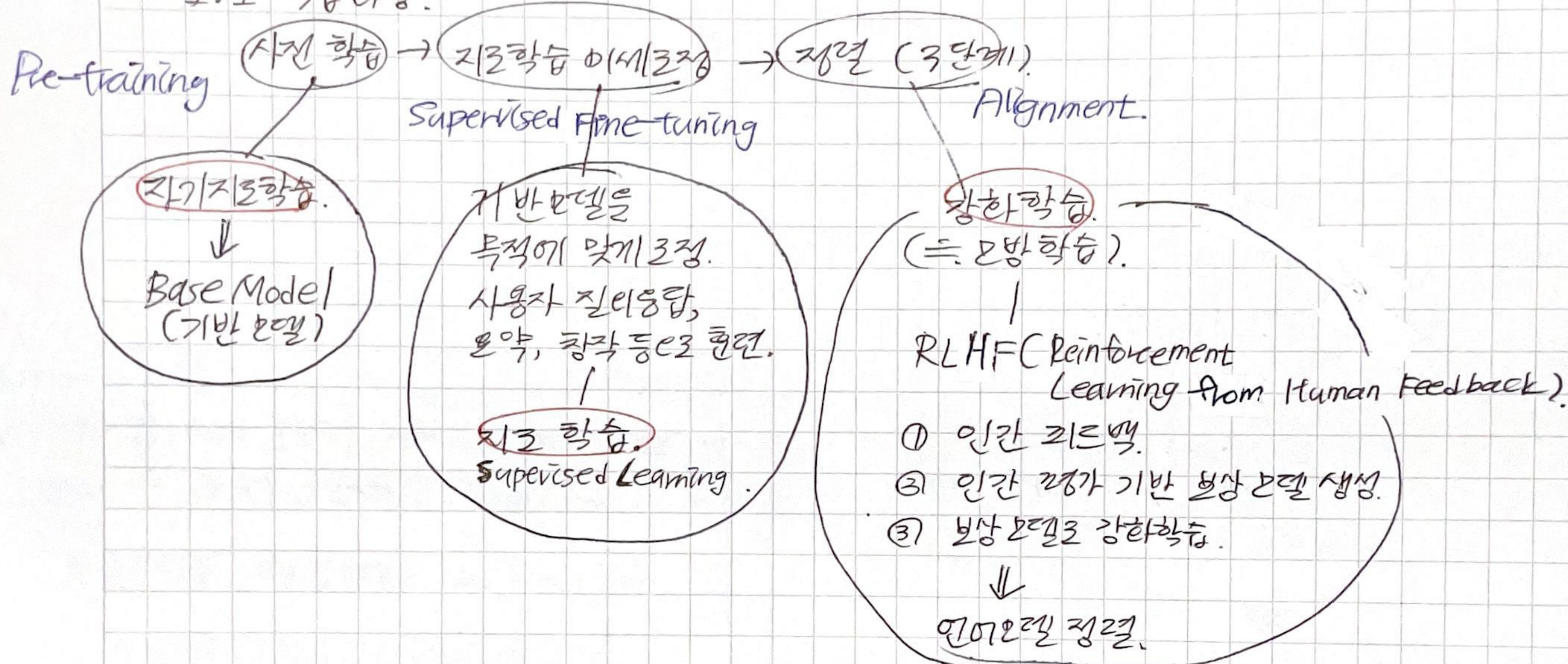
LLM-RAG-GraphRAG.

Continued from page: 생성모델 검색 지식그래프 기반 RAG 구조.

1. LLM (Large Language Machine).

- LLM: Input에 기반해 '통계학적'으로 가장 적절한 output 추출하도록 학습된 모델.
- SLM: 작은 규모, 다룰 수 있는 범위가 좁다. '특정 도메인에 최적화!' → 높은 정확도.

1.1 학습 과정.



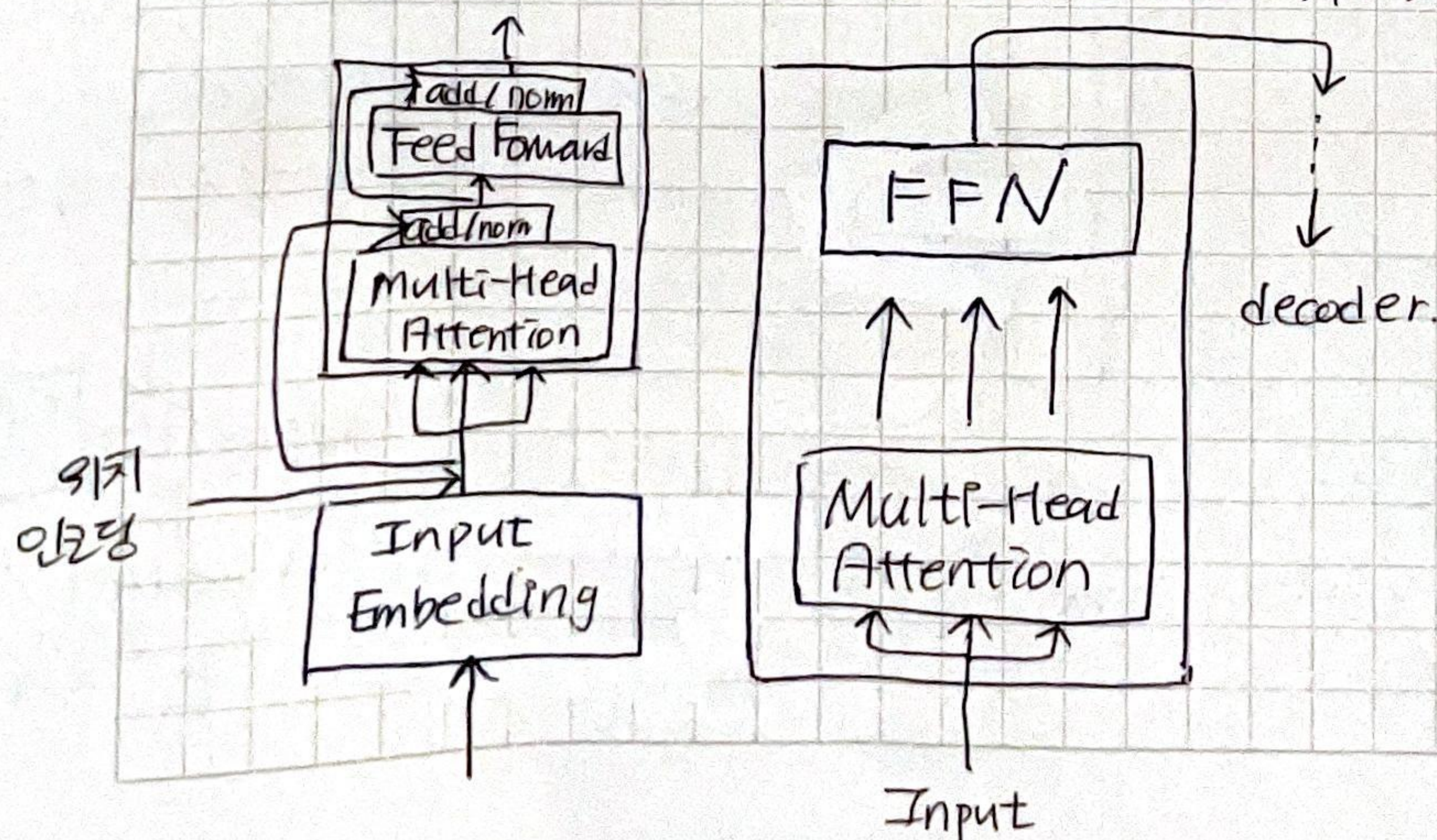
1.2 구조: Transformer & attention mechanism.

- 구성요소: Query, Key, Value (3 벡터). 특정 쿼리에 대해 모든 키에 유사성 (Attention Score) 계산. 이 유사도를 가중치로 사용하여 밸류의 가중합 계산.

- Multi-Head Attention: 어텐션 = 헤드들의 집합.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n) W_O$$

$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$



1.3 한계:

- Hallucination
- 최신성 ↓, 학습 이후 데이터 부족
- 제한 데이터 접근 불가
- 추론 과정에 대한 비가시성.

Continued to page:

RECORDED BY 기록자

최지우.

DATE 기록일

2026.03.04(수)

SIGNATURE 서명

최지우.

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

TITLE 제목

RAG (Retrieval-Augmented Generation). 검색 증강 생성.

Continued from page:

2. RAG.

2.1 RAG 정의: 외부 데이터 검색 결과를 LLM input에 포함시켜 output 생성하는 방식.

user 질문 → Vector Search (유사도 기반 검색) → 검색결과 + 질문

↓
LLM → 답변 생성.2.2 목적: LLM의 단점 보완, 개선
Hallucination.

- ① 비용 효율적인 구현.
- ② 최신 정보 접근 가능.
- ③ 사용자 신뢰 강화.
- ④ 개발자 제이 강하.
(잘못된 참조 수정 등).

(Dense Vector)

2.3 구조: ① 외부 데이터 준비.

문서, DB, API 등 수집 → Embedding Model로 벡터 변환.

← 구조화된 Data → 비구조화된 Data. 코딩한 수리 벡터로 변환.

⇒ 지식 L2b. 구축.

② 관련 정보 검색.

★ 이미 벡터 유사도 기반 검색.

사용자 질문 → 벡터 변환

벡터 DB에 유사도 계산 → 가장 연관 있는 문서 반환. < 거리 기반 검색 >.

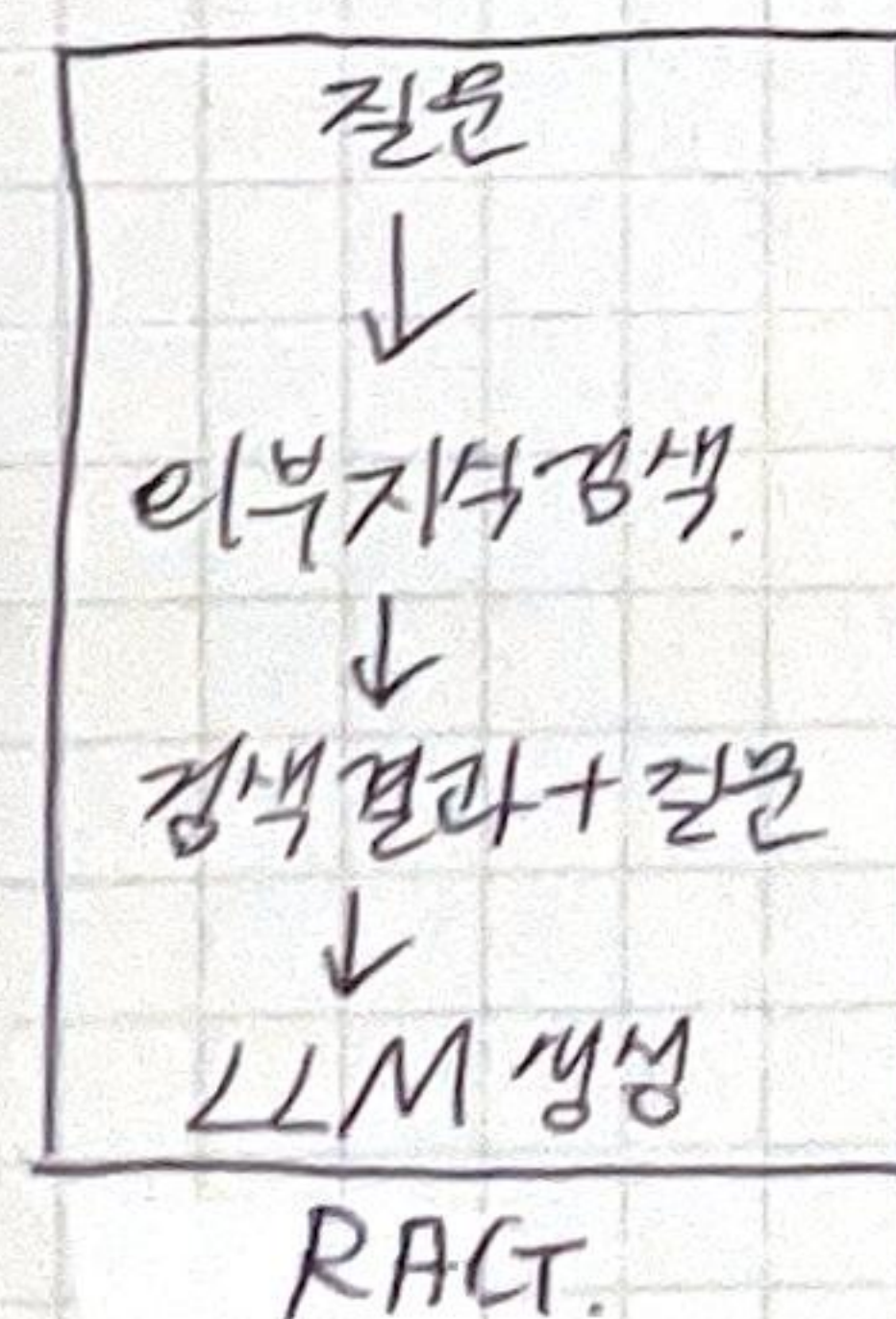
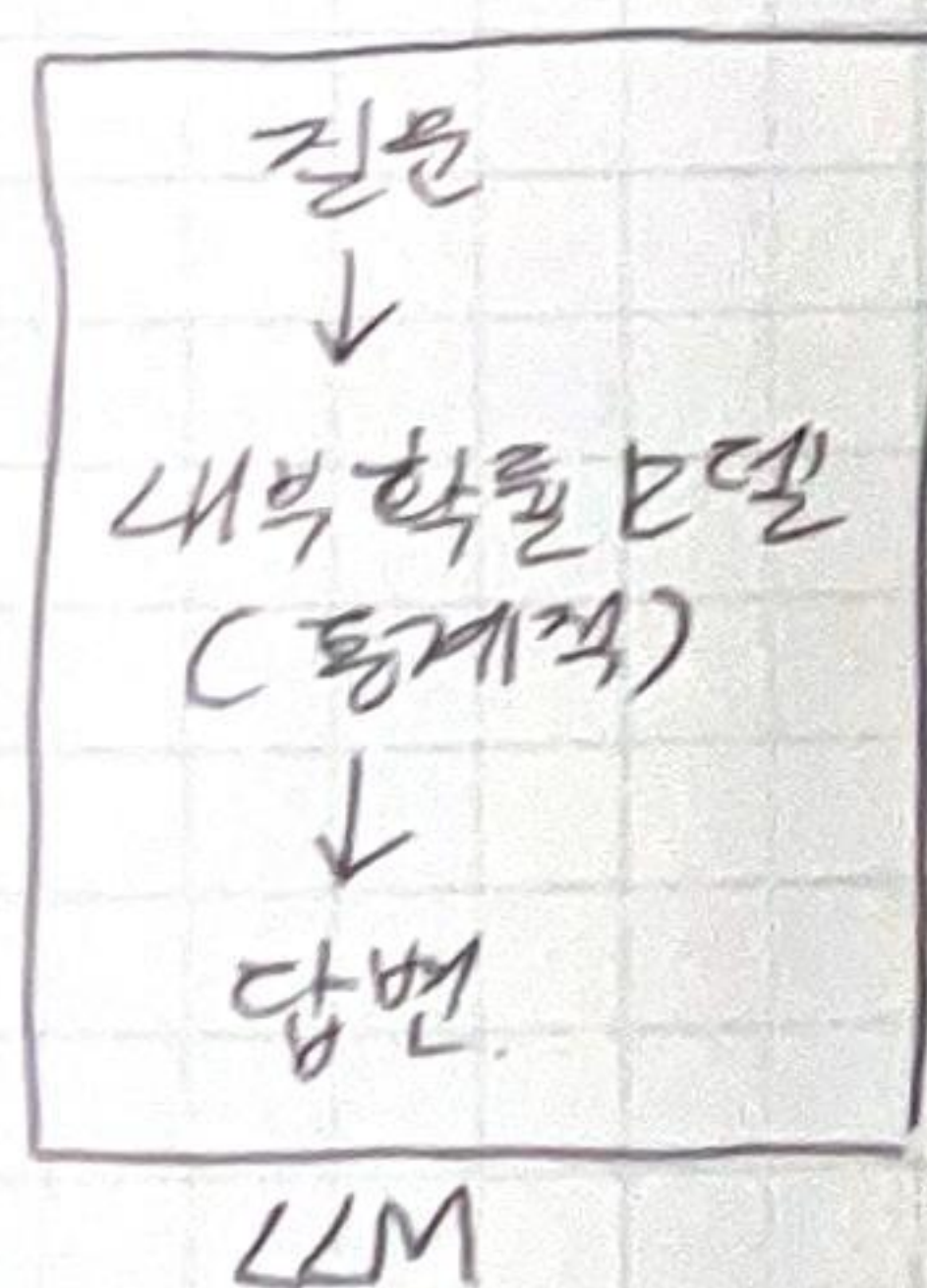
③ 프롬프트 확장.

검색결과 + 사용자 질문.

④ 외부 데이터 업데이트.

★ 모델 재학습 불필요.

2.4 LLM 라이프 사이클.



cf. 시맨틱 검색: "이미지 기반" 검색.

2.5 요약: ① RAG = Embedding + Vector DB + LLM. (벡터 검색 + 생성 모델).

② LLM이 외부 지식을 참조하도록 만들어, 한계를 줄이고 최신성, 신뢰성을 확보하는 구조.

Continued to page:

RECORDED BY 기록자

최지영.

DATE 기록일

2026. 03. 06 (금)

SIGNATURE 서명

최지영.

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

manifest url/FetchWhitelisted '/'
common.js TunnelURL (x)

Continued from page: ✕ Flask의 리로더: 함수수정 ✕, 기

① app.route('/extract-calendar', method='POST')

↑ 해당 주소로 POST 요청 시
extract-calendar() 실행.

def extract-calendar():

이메일제목+본문에서 '일정 이벤트' 추출하여 빈칸
그림 확장 프로그램에서 메일을 열람할 때,
즉시 캘린더 화면을 보여주는 용도.

Flask 서버
내부에서① routing table
등록

[extract-calendar | extract_calendar]

② 요청/응답
handling

[요청: 본문(JSON)
응답: 사용자가 전송한 데이터 → 함수가 완기
전한 상태로

함수의 반환값을

∴ 그냥 파이썬 함수에서 → 외부에서 인터페이스를 통해 접근할 수 있는.

"API 엔드포인트" 함수로 업그레이드 (기능 추가)

• POST /upload : 데이터수집 및 Graph RAG 인덱싱.

Gmail
mailFlask
제출본문제함

① 파일 저장

② mail_latest.txt 생성 (항상 최신 데이터로
③ 그래프 빌드 [mail2json.py 실행, 데이터 쓰기]
graphrag index 실행

• POST /extract-calendar : 전달받은 제목+본문을 분석해
Google Calendar에 등록하기 좋은 JSON 데이터로 변환. (-convert-to-calendar-text)

• POST /run-query : 질문 받아와 job-id 반환. 실제 답변 생성은 by (threading.Thread) 의 멀티
-asunc 답변 도래 기다리라고 연결 끊기지 않도록 하기 위함.

• POST /run-query : 질문 후 답변 도착할 때까지 대기. 간단 텍스트 or 빠른 응답 가능할 때 사용.

• GET /graph-data : mail2json.py가 생성한 노드, 엣지 데이터를 JSON으로 변환.

src/parquet/input/mail_latest.txt → src/json/graphml/data.json

Continued to page:

여기서!

RECORDED BY 기록자

최지규

DATE 기록일

2026.03.09(월)

SIGNATURE 서명

최지규

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

Continued from page :

cf) src/parquet/settings.yaml ← GraphRAG 인덱싱 설정파일.

- GmailWeaver/src/apps-script에서
- GmailWeaver/에서

clasp push.

python src/app.py

- * Serving Flask app 'app' app 자체 실행
- * Debug mode: off '유명도'로 실행
- * Running on all addresses (0.0.0.0) 모든 네트워크 호스팅 OK
- * Running on http://127.0.0.1:8030 30번
- * Running on http://223.194.139.125:80 외부 망 접속용.

- 3110 크기제한 텍스트 수신 (약 142903 Byte).
- 그레프 쿼리 ← (src/mail2json.py)
15도 = 48, 21도 ← (src/parquet/input/mail-latest.txt)
- GraphRAG 인덱싱.

↓
Error: Incorrect API key provided
* Open AI 키 새로 발급받아야 함! 시급!

Python, JS 네이밍 통일

name: ~~내부에서만 사용~~ Internal use
e.g) app.py
util/...js

⇒ README.md 수정.

시스템 구조. → 라디오 추가
내용 만들기

settings.yaml 수정 → feat
b 이 push.
↓ merge

이후 작업

중등그라미. [abc] 중등그라미 생성 09(월) <TO-DO List>

↳ 크래프 클릭시 ~~시작~~

util/naming
수정.

★ 그레프 수정 ★

애드온 : 간단.
[유행] : 가볍게...

• 논문 2개 ↑ (지식그레프 관련)
↳ 정리.

• 시스템 구조.
• 전체 실행.

• README.md

① API 키 발급.

↳ LLM 허가사항.

② 라게이 promps/
크롬캐스트 수정

③ 질의응답시
컨텐츠 C. → 유행생성시 → 유행생성
30도이상 대체생성

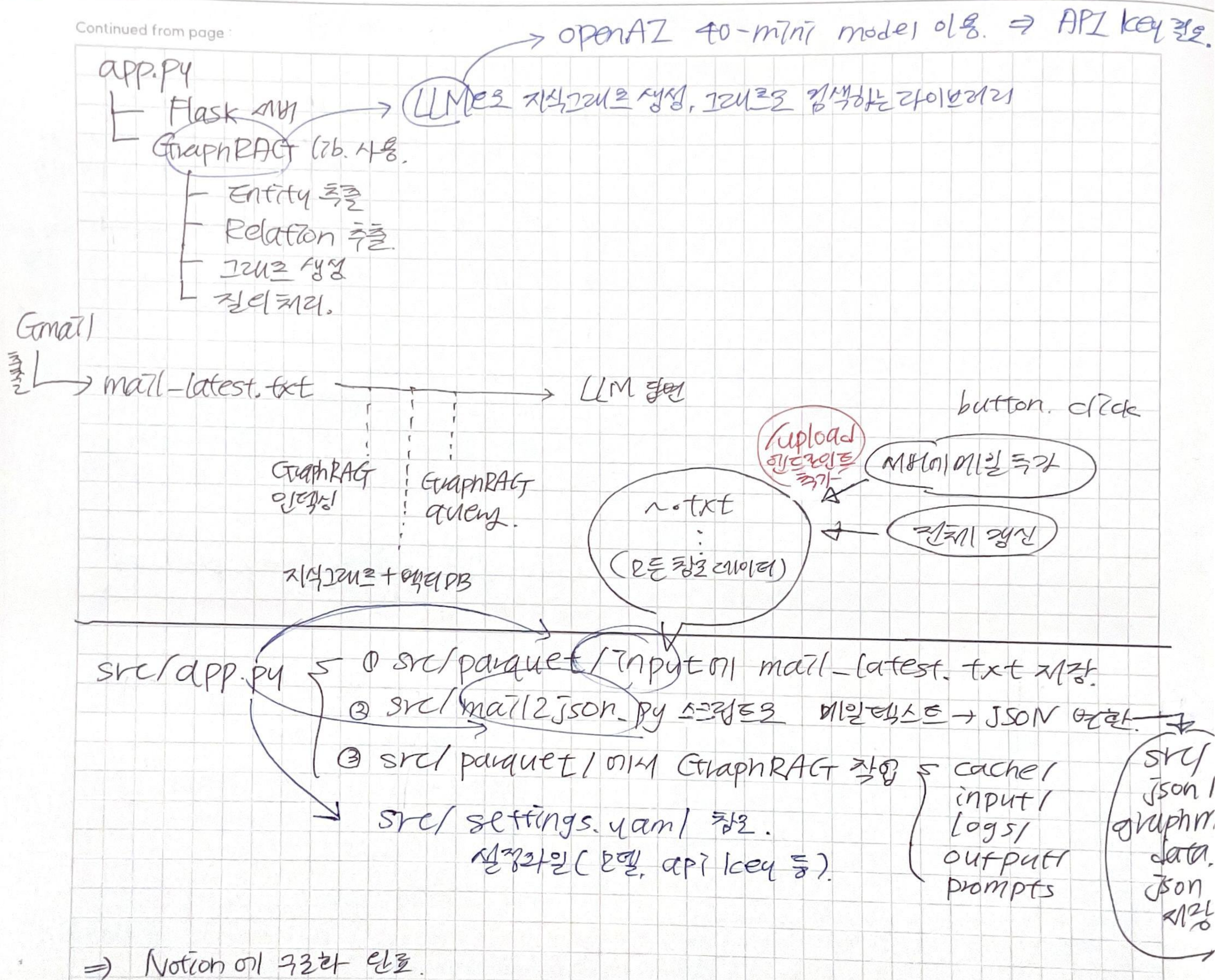
RECORDED BY 기록자

WITNESSED BY 확인자

TITLE 제목

GraphRAG 크라이프라인 workflow.

Continued from page:



Continued to page:

RECORDED BY 기록자

최지우

DATE 기록일

2026. 03. 11 (수)

SIGNATURE 서명

최지우

WITNESSED BY 확인자

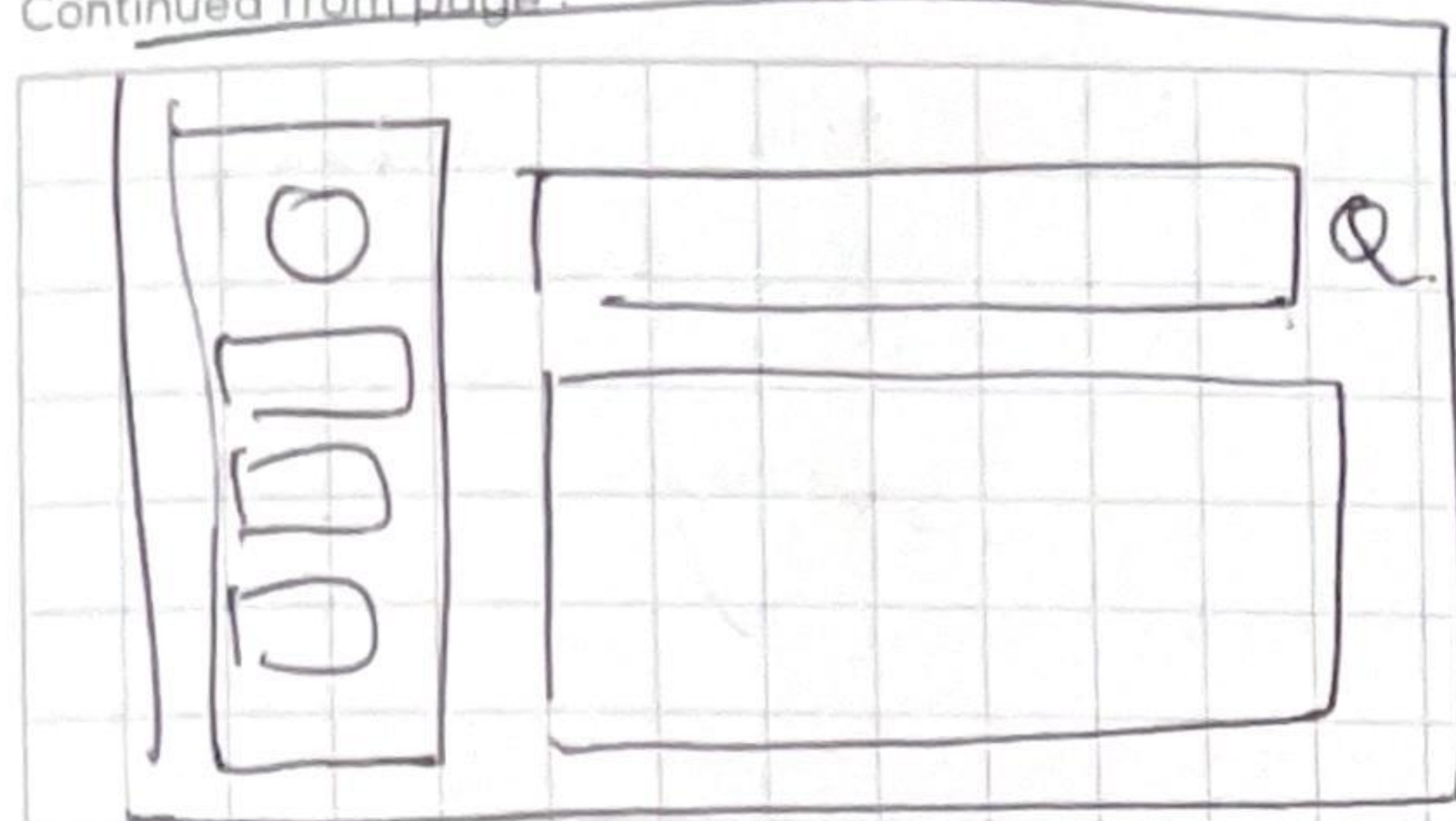
DATE 점검일

SIGNATURE 서명

TITLE 제목

UI prototype 형식.

Continued from page :



즉시별로 카테고리
 스타일 메일 스타일
 라벨로 분류하기

시간
 사랑
 강도
 시간, 알람

여러개의 메일 찾기

하루의 메일 찾기
 (e.g. 대한항공 최근 예약내역 보여주기)

① 검색

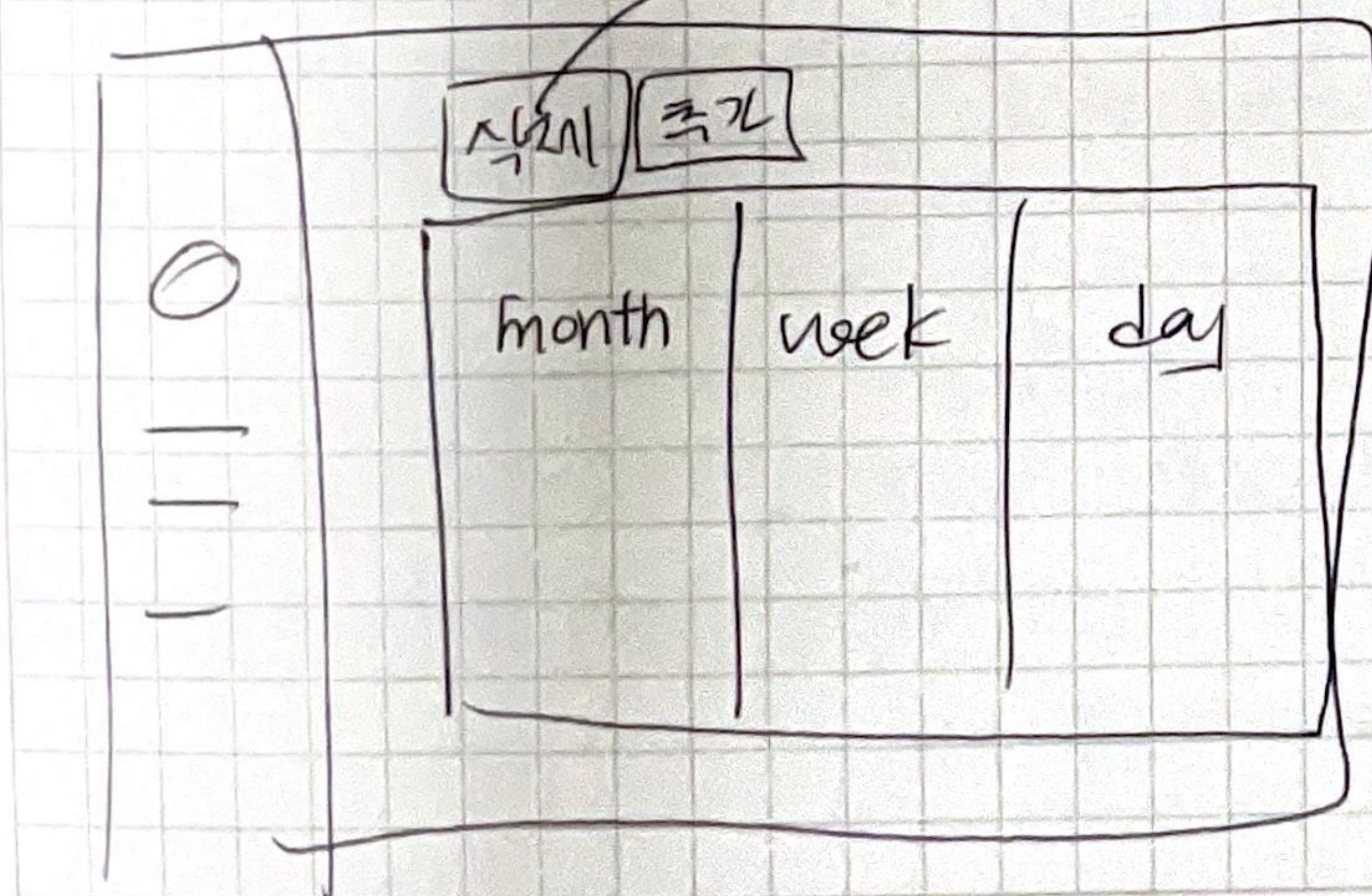
② 분류

④ 통계



상세 보기

③ 캘린더



Continued to page :

RECORDED BY 기록자

최지우

DATE 기록일

2026.03.13 (금)

SIGNATURE 서명

최지우

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

TITLE 제목

첨부파일(PDF, Docs) → 텍스트로 변환

Continued from page :

* 검사함. 첨부파일 확장자(-jpeg, -png, ...)

↳ 이미지 인식까지 가능하도록?

Client

/upload로
메일 본문 +
첨부파일.

Server

- attachments = data.g
- path 이용해 파일 read.
- mail-latest.txt 생성
- subprocess로 mail2
- 그라운드그라운드 인데싱!

~~1. 메일 본문 클러스터링?~~

- 26.03.13 일의 내용 정리
- 라벨 클러스터 mail-latest.txt 생성 + 첨부파일

✓ Client 측, (메일 전송)

attachment: [] 형식

~~전송하는지 확인~~ **확인**~~manifest URL 확인~~

<테스트 코어터 비교>

Base64

Client

Add-on.

내

- 스트림 검색
- 메시지 식별
- 첨부파일 목록화
- "bytes" 확보

Server

- 인덱싱 X
- 파일 하나 받아 텍스트 추출

Input

filename
mime
base64

Output

text
note

- ↳ Path 보라 파일 크기 약 33% ↑
- ↳ 다룬다 라벨에서 CPU 부담 ↑
- ↳ 한때번에 전송시, 서버메모리 과부하 우려.

Path

"path" 경로 자체를 그대로 가져온다는 방식.

* 서버 컴퓨터의 HDD에 물리적으로 저장되어야.

Client

Server

parquet/input/attachments/

서버에 로컬 저장소에 저장한다.

~~* data-base 64.~~데이터 → 텍스트 전송을 위해
(binary 64진법 text, 2 인코딩)JSON&HTML 같은 텍스트 기반 형식에
첨부파일 삽입하기 위함.

Hybrid

- Gmail API로 클러스터링, 첨부파일 (Base64) 받아와서 임시 디렉에 그대로 저장.
- upload/에서 파일 내용(Base64) 대신, 저장한 파일의 경로(Path) 전달.
- 전송받은 서버는 경로 추적 → 히스토리 추적. 처리 끝나면 보관.

Continued to page :

RECORDED BY 기록자

최지규

DATE 기록일

2026. 03. 16 (월)

SIGNATURE 서명

최지규

WITNESSED BY 확인자

김기재

DATE 점검일

2026. 03. 20 (수)

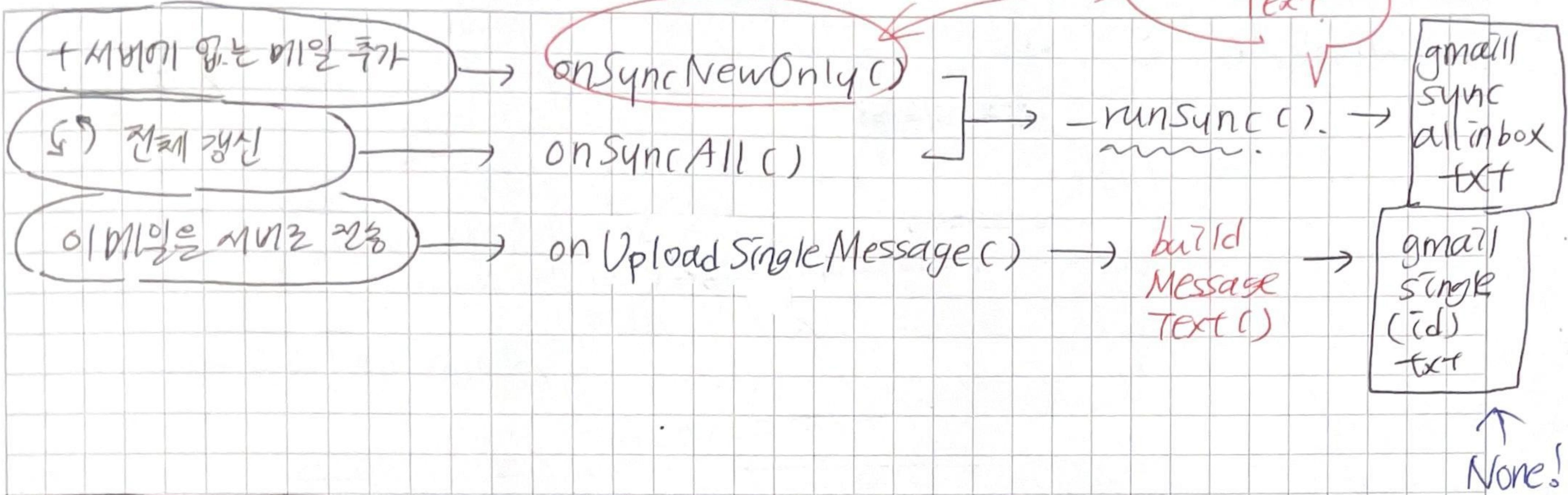
SIGNATURE 서명

김기재

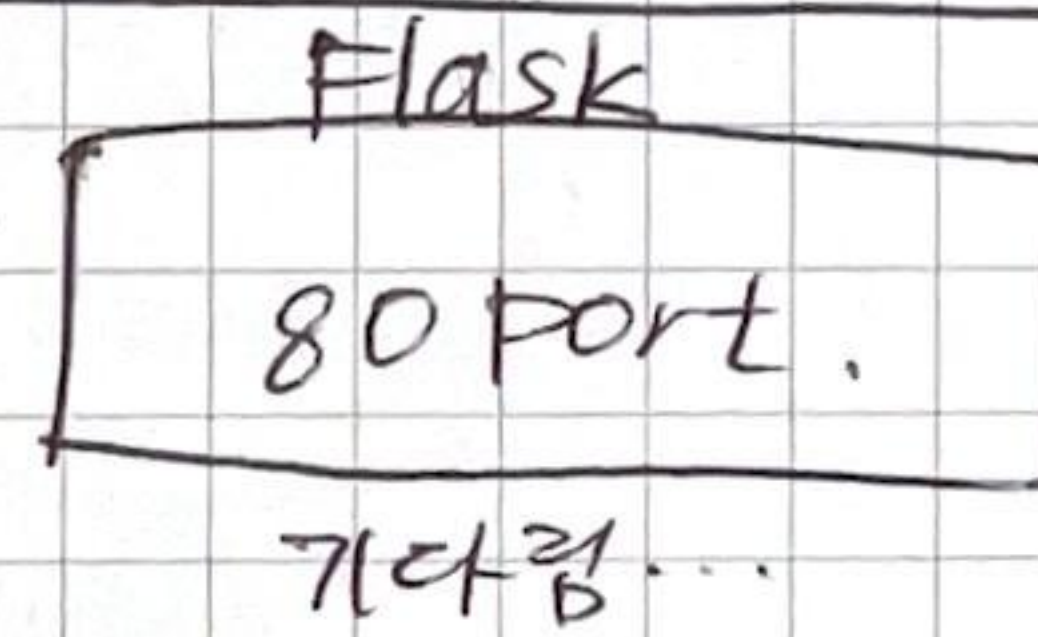
TITLE 제목

GraphRAG 위아래의 작업.

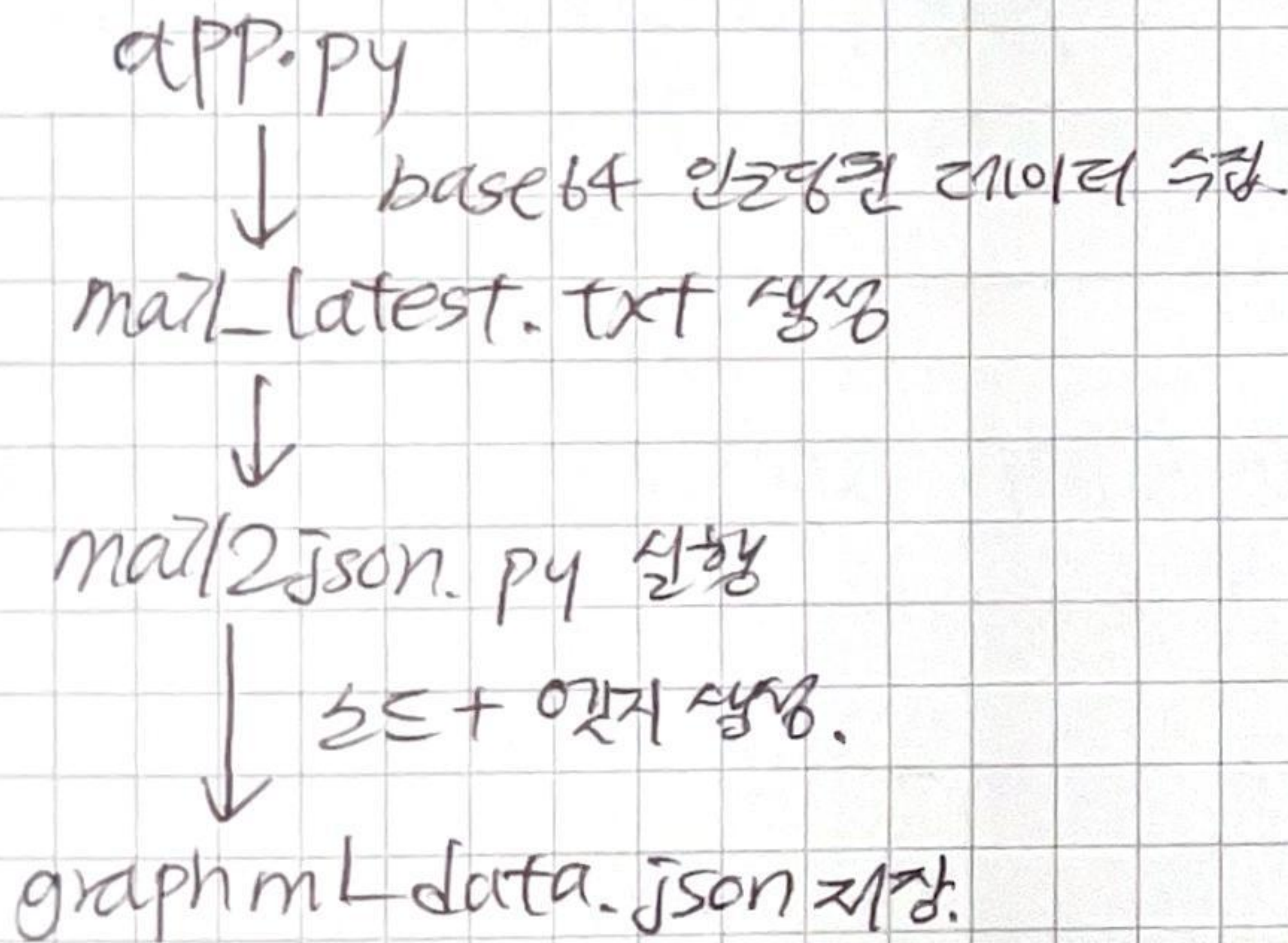
Continued from page :



app.py → run 80 port



Pipeline



Graph Schema

지식 그래프에서
구분, 엔티티 유형, 속성, 엣지 등
정리된 설계도 (메타데이터)

Nodes Type

- Email
- Subject
- Person
- Label
- FILE

(id, type, description, properties (속성))
: 메일 id, 제목,
: 라벨, 본문,
: law--

Edges

- 1) 사람 → 메일
- 2) 메일 → 제목
- 3) 사람 → 제목

Context
생성

① source = 주체	
② type = 종L	Relates To...
③ target = 목적어	Email: ...

Continued to page :

RECORDED BY 기록자

최지영

DATE 기록일

2026.03.20(금)

SIGNATURE 서명

최지영

WITNESSED BY 확인자

DATE 점검일

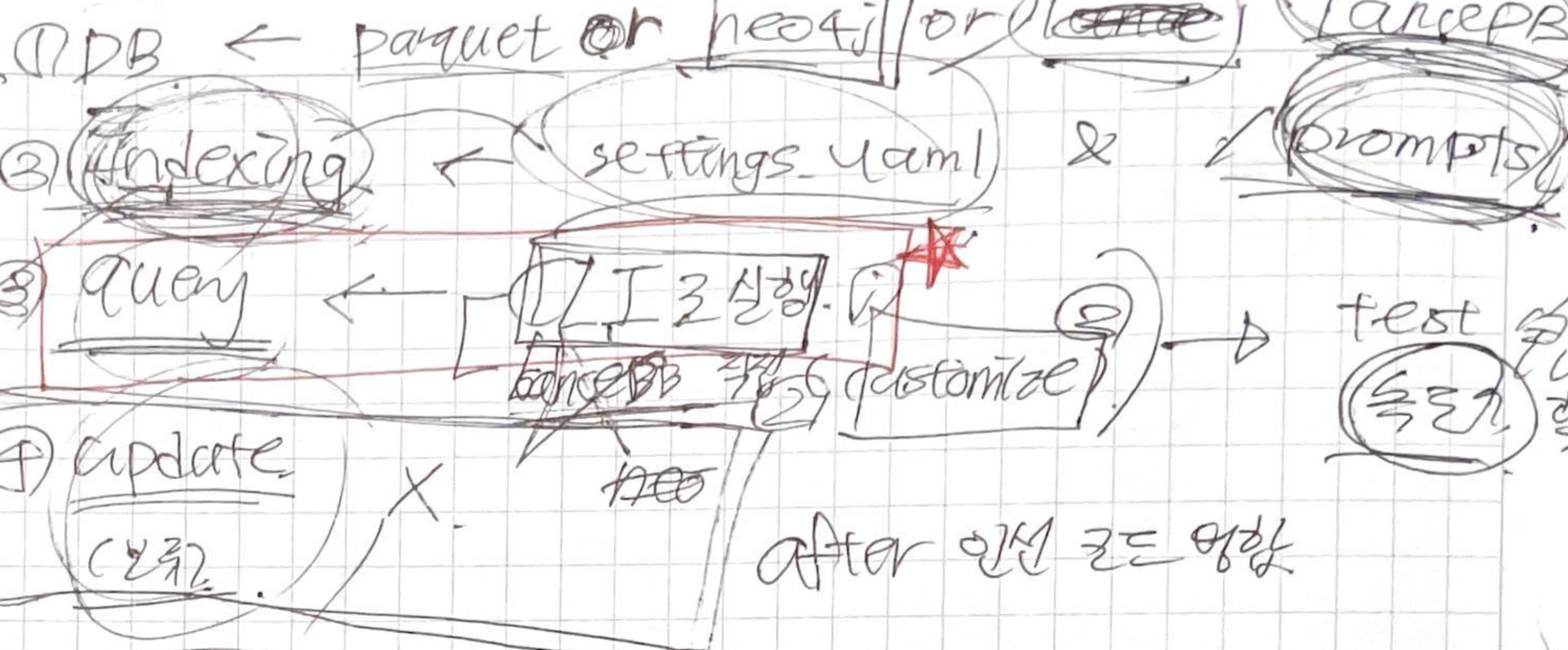
SIGNATURE 서명

TITLE 제목

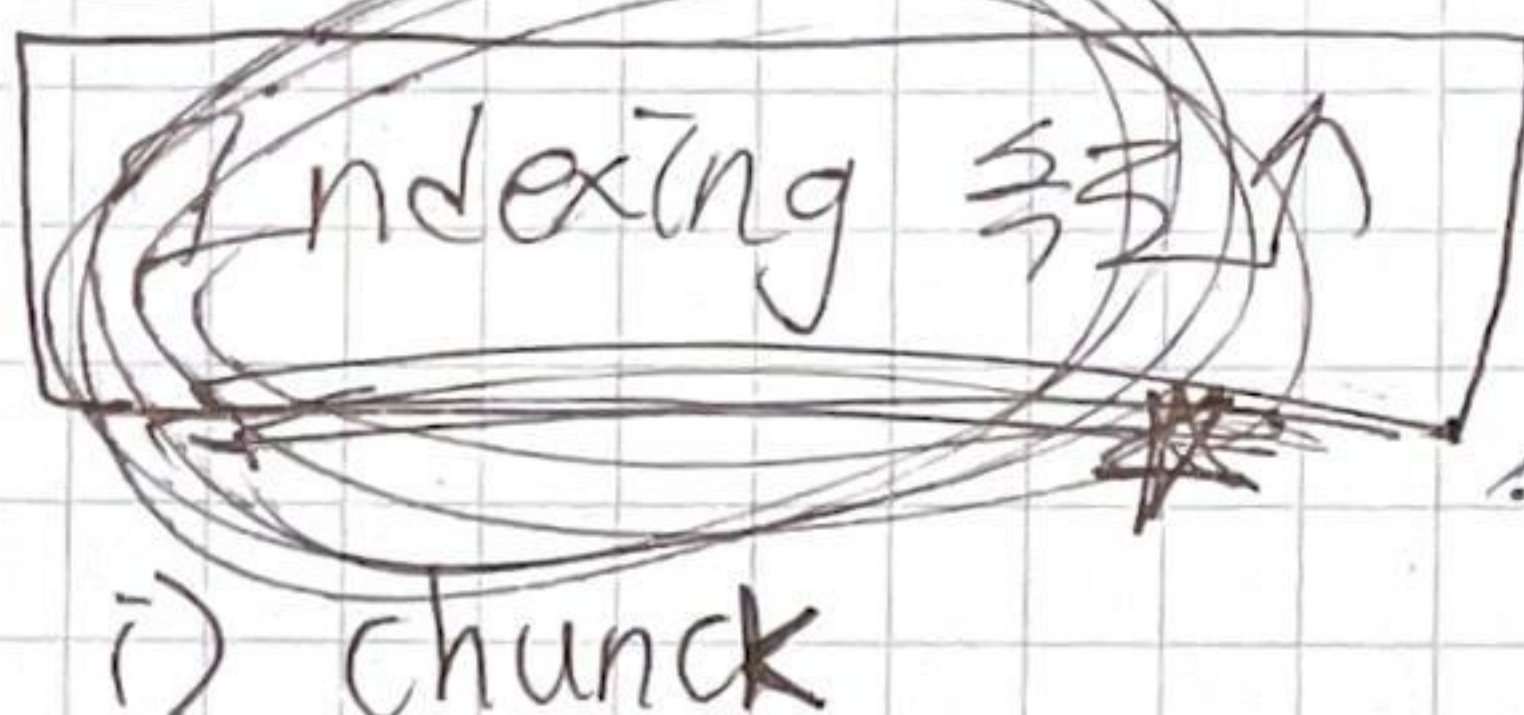
GraphRAG Indexing 흐름 향상. ajinh.

Continued from page

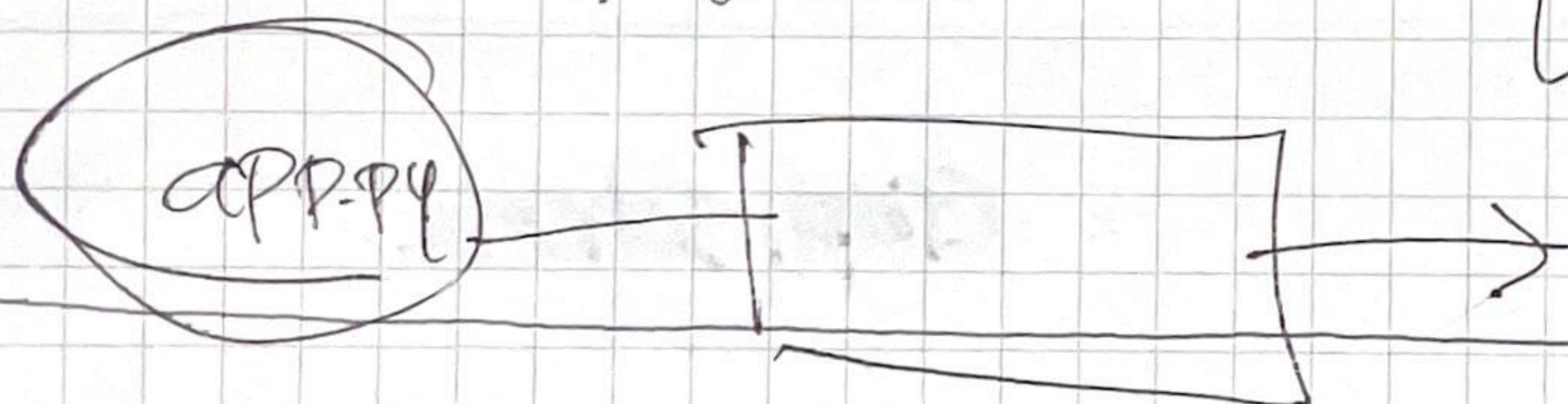
GraphRAG.



Task.



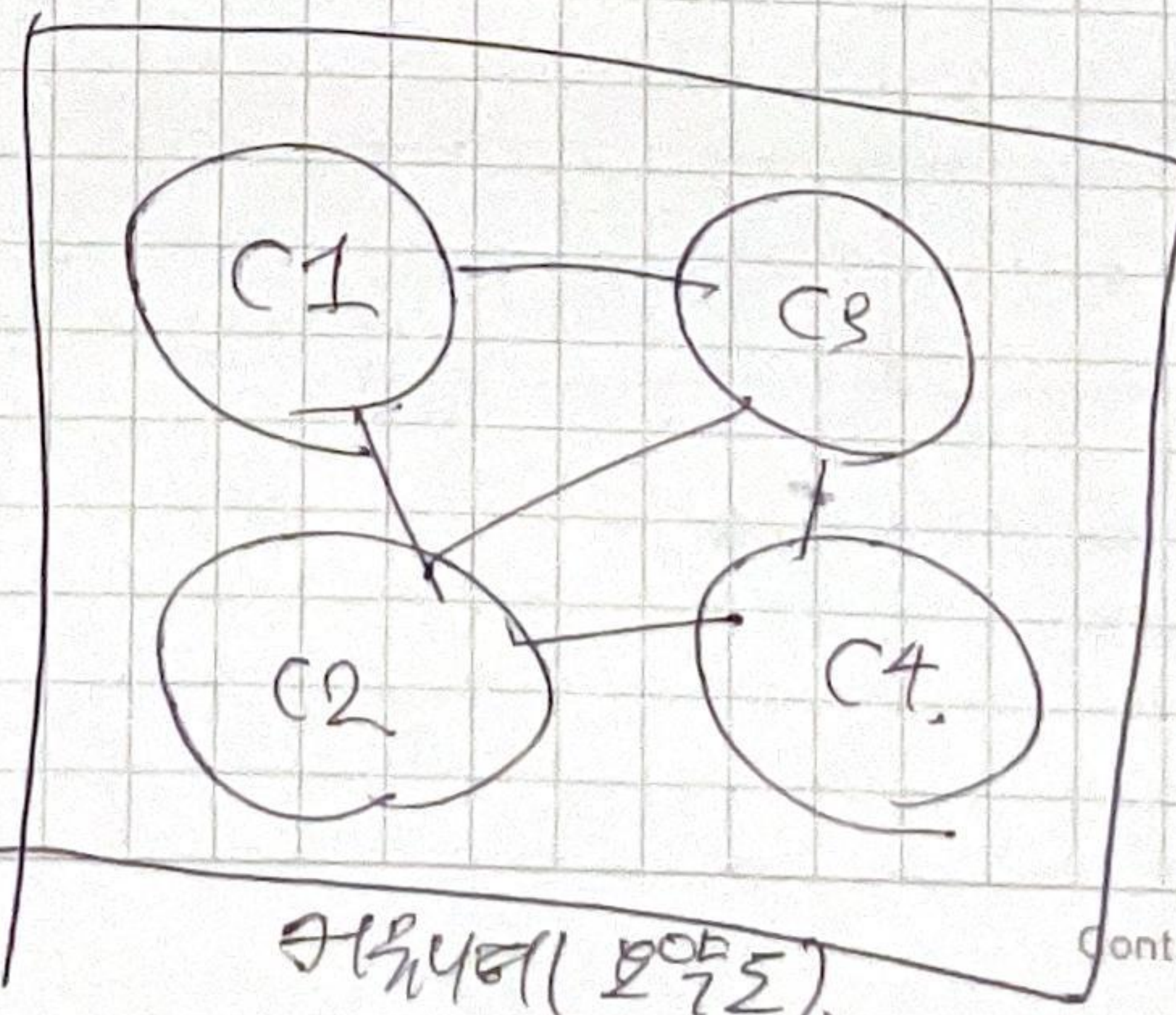
Prompts (알아보기, 삭제, → 검색, 질의 영향)
settings.yaml



★ GraphRAG 최신 버전 활용 위해 settings.yaml 재설정.
PY 버전도 확인함.

특정 global 객체

GraphRAG 그룹화 → 그룹화 도출 → 그룹화 도출 → 그룹, 항목.....



Continued to page

RECORDED BY 기록자

최지욱

DATE 기록일

2026. 03. 20(금)

SIGNATURE 서명

최지욱

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

TITLE 제목

프로젝트 개선 위한

Task List

지금은 query

Continued from page:



APZ? 문장

★ Gmail 특화적인
기작 그래프

개선관...

util/neo4j-query.py

결과: 즉각 & (2) ↓

인사... 직접 python 코드 짜.

DBMS

manage

neo4j

DB

parquet

결과...

목표는 인덱싱 시간 단축!

클라우드 환경에 대한 답변 관리 향상.

- parquet / 내부 파일 (산출물) 조사.
- settings.yaml 과 prompts / 가 어떤 영향 미치는지?
- 그중 하나 등이 어떻게? 진행되는지? 개선 방안은?
- GraphRAG 버전 & settings.yaml & Python 버전과 가상환경.

+ ... 그리고 클레이블 코드...

입력 → 중간 산출물 → 최종 산출물

- gmail.js 주석.
- 수정사항 정리 + 개념 (공백)
- git pull upstream main
- 코드 리뷰 PR.

그래프 도입

Continued to page:

RECORDED BY 기록자

최 지 역

DATE 기록일

2026.03.20 (금)

SIGNATURE 서명

최 지 역

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

TITLE 제목

prompts 분석.

(비용 고려사항).

* UZID = Universally Unique Identifier.

Continued from page :

local-search-system

목적: 한락제기. 딱 제공된 데이터 레이블 내에서만 응답하도록 제한하는 엄격한 프롬프트.

장점: 즉응성 (대부분 레이블링 X) → 한락제기.

단점: • generic. 지메일 특화 X. 지메일 특성 파악해 더 적합한 리에 부여해야 함.

100. [즉응성] 프롬프트로 인해 대략 일리언 즉응도 불가하다? ⇒ 그래프 즉응 불가!

• 같은 대형 LLM 모델을 다른 용도로 활용해 성능 저하.

• {response-type} & {context-data} 스키마 정의 X. LLM의 더 강력한 해석 가능.

↓ 텍스트 데이터.

e.g. 한국어 응답 (app.py?)
[지메일-상세-결과] 작성

① 개체 레이블 = id, entity, type, description.

② 관계 " = id, source, target, "

③ 리포트/주장 " = id, entity, claim, status.

(상위 수준의 요약 정보에서 추출된 주장).

• All Information 강제

↳ 응답 길이 ↑, 토큰 사용량 ↑, 불필요한 정보 포함 ↑.

★ ~~이제~~ ~~바탕~~ ~~클릭~~ 질의 (종료)

질문
바탕 클릭.

전송

onSmartSearch()

접속

/run-query-async

① full message
worker() ② payload 전달
↓ ③ 전송

onSmartSearch() ← query.js.

사용자 입력 추출, 정제 (공백제거, 빈값 X).

↓
서버 /run-query-async 접속.

```

{ method: "post",
  contentType:
  headers:
  payload: { message: query
            resMethod: local
            resType: structured
            }
  }
  비동기 jobID 가져오기.
  update (jobID 유무 따라).
  
```

LLM

① 검색
② Context 조합
③ LLM 전송

run-graphrag()

응답 반환

success 부분만 전달

<서버>

Continued to page

RECORDED BY 기록자

최지우

DATE 기록일

2026.03.23 (월)

SIGNATURE 서명

최지우

REC

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

WITN

TITLE 제목

Need to know (문, 임베딩, 컨텍스트창등).

Continued from page :

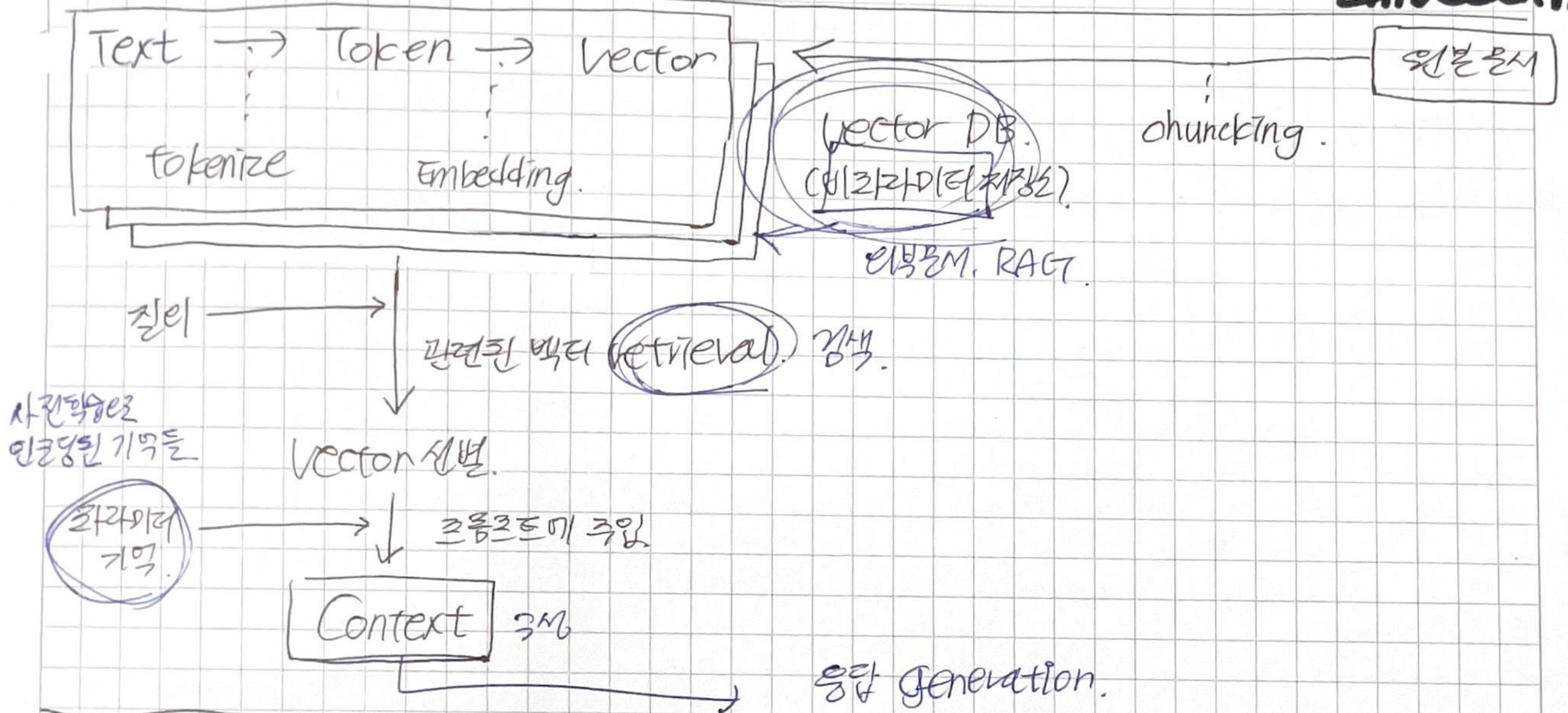
Community-report.

GraphRAG가 전체 그래프를 한 번에 읽을 수 없으므로, 비슷한 것끼리 묶어, 그 요약본(리포트)을 미리 작성한다. 필요 시 이 리포트들을 찾아 답변하는 식이다.

목적 : 커뮤니티 내의 지체 목록, 관계, 움직임 → 종합 리포트 작성.

decision-makers (의사) - 커뮤니티 & 그 잠재적 영향을 알리는 역할.

Embedding



webapp URL.) 공제 → clasp push
Tunnel URL
• "local"로 지정 후,
local-search-system.txt
개입하여 테스트. 수정.
• 데이터 추출하도록 entity-extraction
필요한
• 어떤 엔터티가 필요할지?
• 한글, 영문, PPT 형식파일 txt도 클로드
이러
• 인풋 파일을 어떻게 개입해야 인풋싱가?

할일... until 주할 ☆

- ① 데이터 추출 prompt
 - ② ~~데이터~~ 클레임 추출
 - ③ 커뮤니티 리포트
- ☆
- ④. FigJam 프롬프트 전체 관계.
- ⑤. 클레임 생성 테스트
- 클레임에 비교. → 분석하.

Continued to page :

RECORDED BY 기록자

최지우

DATE 기록일

2026.03.23(월)

SIGNATURE 서명

최지우

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명

TITLE 제목 GraphRAG Workflow & Community.

Con

~~Community report.~~ Pipeline

GraphRAG Indexer.

- Loading Input. ← mail_latest.txt
- create_base_units. → text_units.parquet
- create_final_documents. → documents.parquet
- extract_graph. → entities.parquet & relationships.parquet
- finalize_graph. → $\hookrightarrow$ 그래프, 보강.
- create_communities. → communities.parquet
- create_final_text_units. → text_unit.parquet 보강
- create_community_reports. → community_reports.parquet
- generate_text_embeddings. → lanceddb/

Community.

- id
- human_readable_id
- community
- level
- parent
- children
- title
- entity_ids
- relationship_ids
- text_unit_ids
- period
- size

커뮤니티 고유 식별자.

사람이 읽는 ID.

커뮤니티 번호.

계층 레벨 (0: 최상위)

부모 (상위 커뮤니티) (-1: root)

자식 (하위 커뮤니티)

이름 (e.g. "title": "Community 11")

커뮤니티에 속한 엔티티들 (UUID)

커뮤니티 내 관계들 (UUID)

근거가 된 텍스트 청크들 (UUID)

인덱싱 날짜

엔티티 수

계속됨

(4명)

동일.

Continued to page

RECORDED BY 기록자

최지우

DATE 기록일

2026. 03. 30(월)

SIGNATURE 서명

최지우

WITNESSED BY 확인자

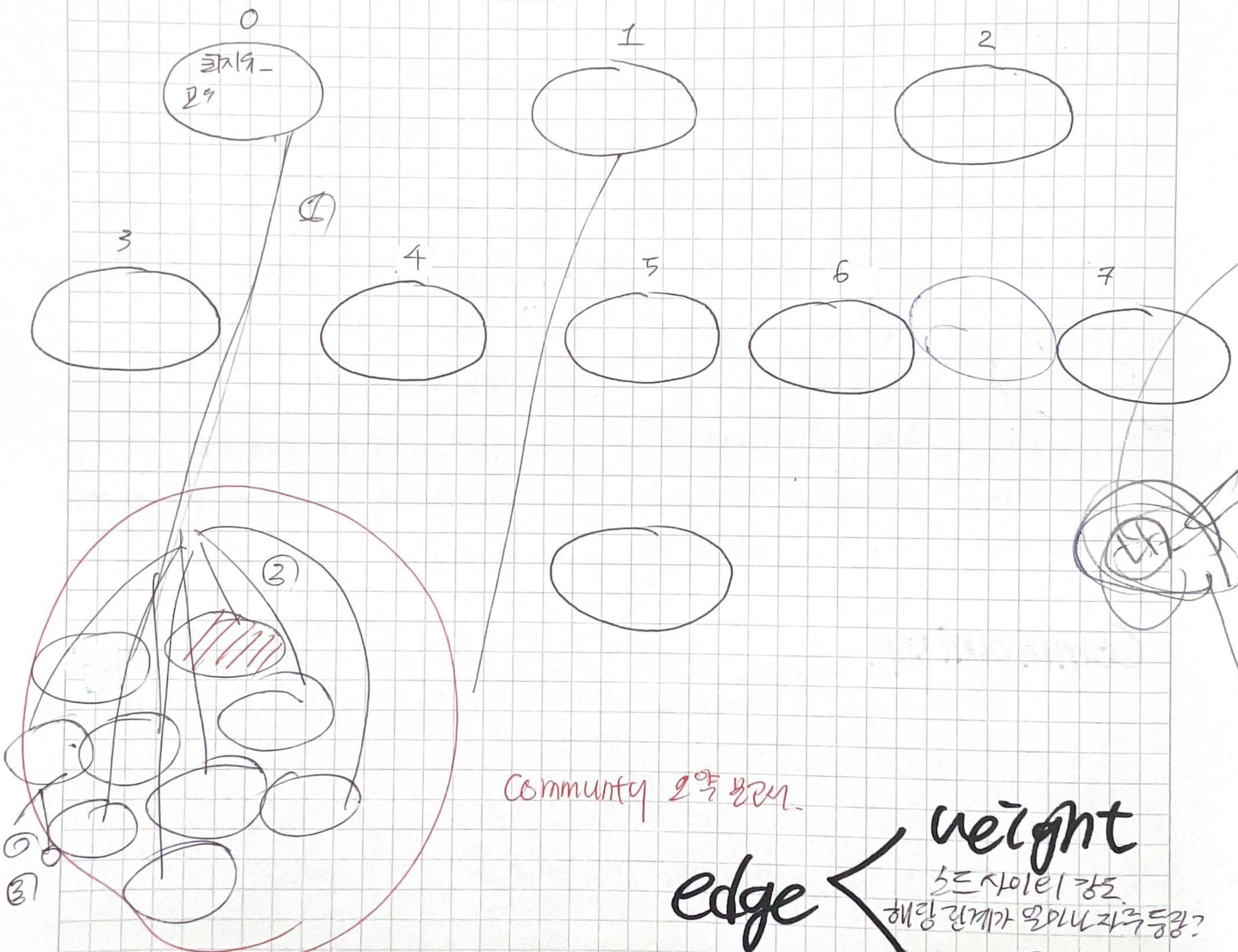
DATE 점검일

SIGNATURE 서명

Community reports.

Continued from page :

Community_reports.

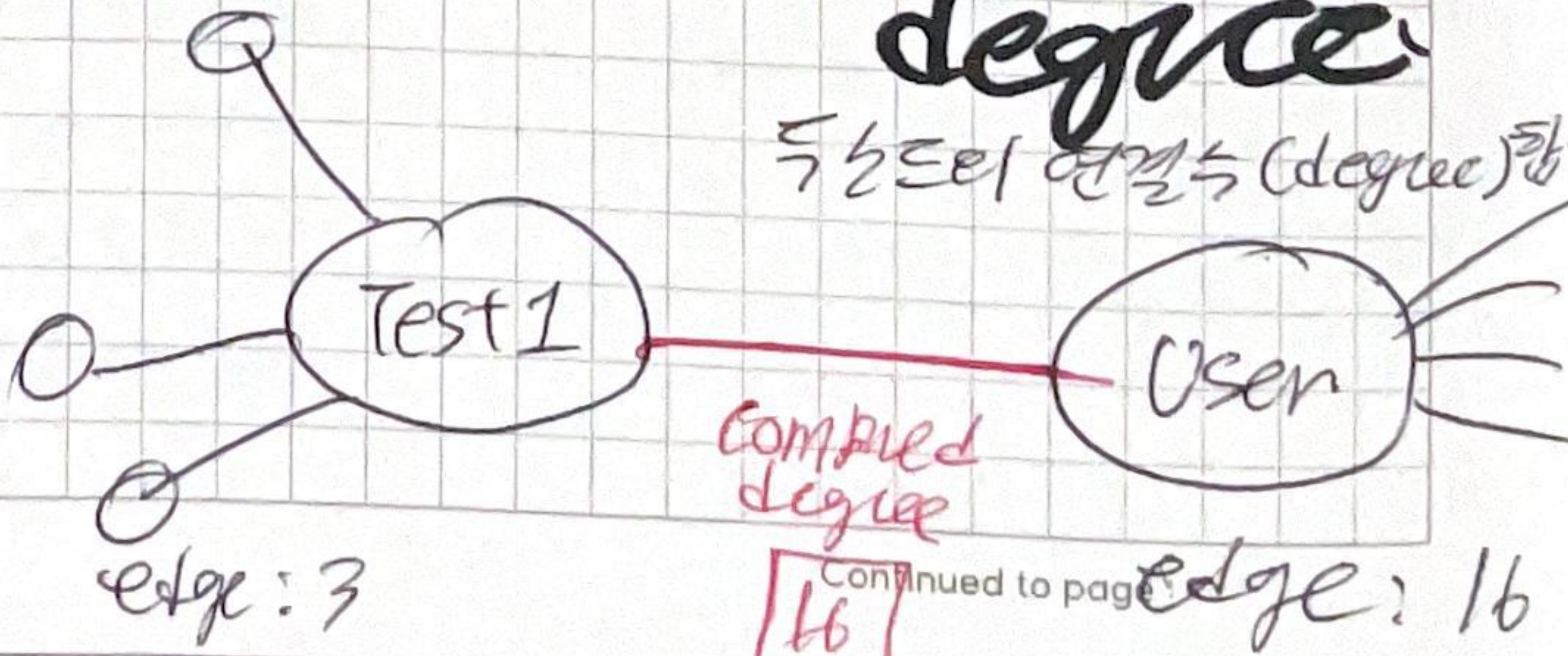


Community 2월 분기

weight
노드사이의 강도
해당 관계가 얼마나 자주 등장?

Combined degree
두 노드의 연결수 (degree) 합산

level 0:
level 1:
level 2:
계층적 클러스터



Combined degree

16

Continued to page

RECORDED BY 기록자

최지우

DATE 기록일

2026.03.31 (화)

SIGNATURE 서명

최지우

WITNESSED BY 확인자

DATE 점검일

SIGNATURE 서명